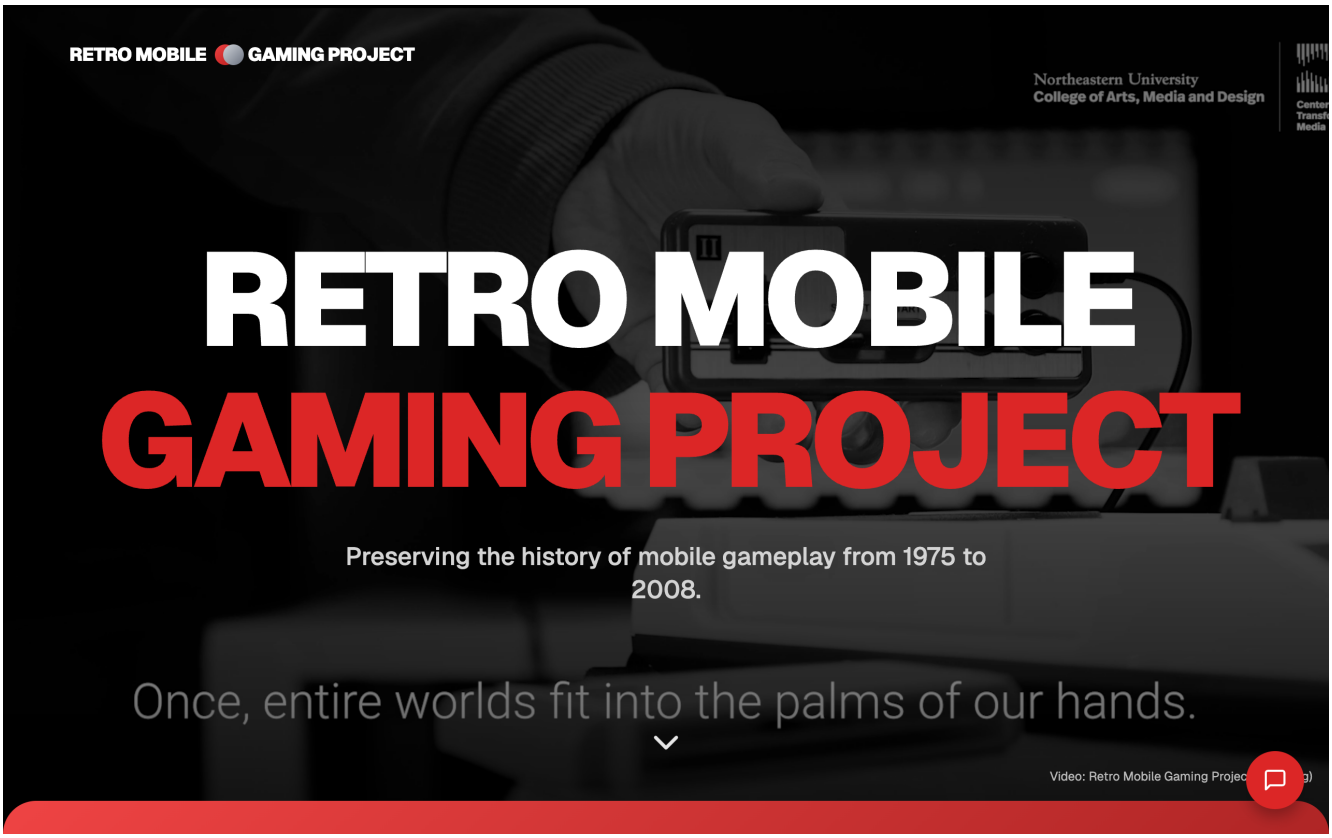
RETRO MOBILE GAMING PROJECT

Front page take-home

A searchable archive page for 45 RMGP games, served from DynamoDB through a read-only AWS Lambda API, with a collection chatbot.

Candidate	Ahmed Khan
Role	Software Developer Research Assistant, Center for Transformative Media
Date	November 2, 2026
Repository	github.com/khanbilal-devop/rmgp-assement (shared with CTM reviewers only)
Time spent	4 hours



The hero section, with RMGP's video and the line on preserving mobile gameplay from 1975 to 2008.

Summary

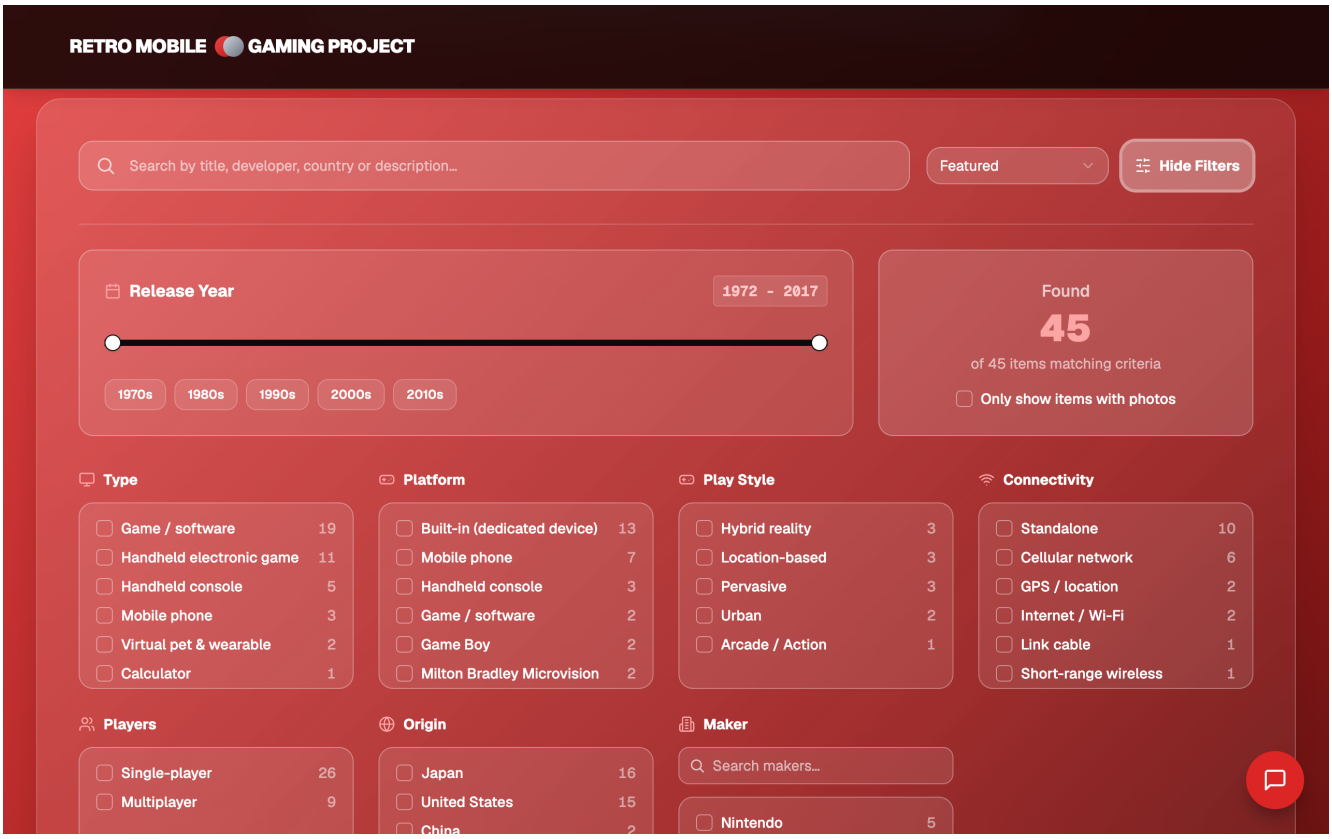
I built one front page in the style of rmgd.org. It has a hero video, a **From the Archive** section for browsing 45 real games from the RMGP collection, and an About section. Visitors can search by title, filter by device, decade and six other fields, and open any game in a detail view without leaving the page.

The games come from a small DynamoDB table in my own AWS account, on the free tier with on-demand capacity. The page never talks to AWS directly. It calls a read-only Lambda through a Function URL, so no AWS keys exist in the code, the repository or the screenshots. A simple chatbot answers questions about the collection and says so when something is not in it.

Part	What was delivered
Front page	React (Vite) page: navbar, hero video, From the Archive, About and the collection guide chat
Data	aws/games.json: 45 games, the same 22 fields on every record, credited to RMGP
Database	DynamoDB table rmgp-games (us-east-2): partition key decade, sort key device#id
API	aws/lambda_handler.py: search, every filter, sort, 8-per-page paging and live filter counts
Seed	aws/seed.py: loads the JSON with boto3 and checks the item count matches
Tests	90 page tests (Vitest) and 72 API checks against the live table, all passing
Chatbot	Keyword-based, no AI model and no keys; answers only from the API

The page

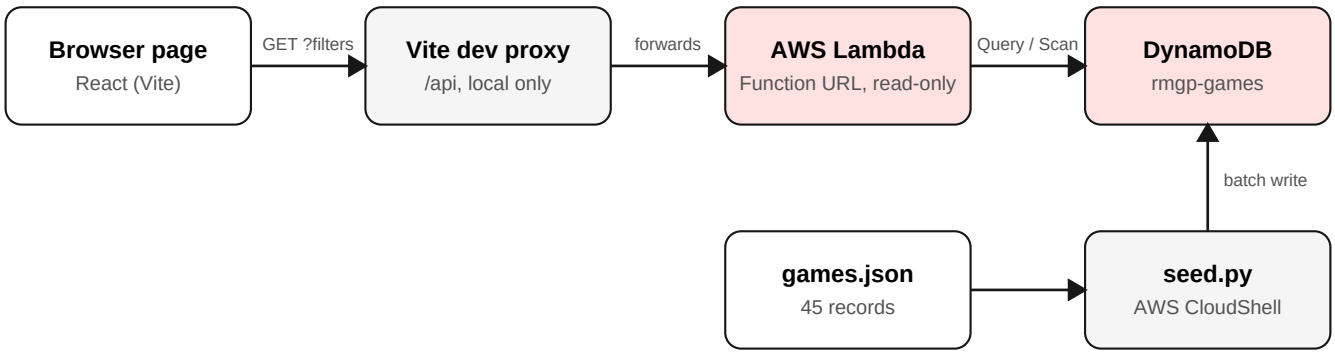
Section	What it does
Navbar	Fixed bar with the logo only. Clear over the hero, dark and blurred after 60px of scrolling, as on rmgd.org.
Hero	RMGP's video, muted, looping and inline, with the brief's line. The title fades out after about four seconds, as on the live site.
From the Archive	Title search, sort, a year slider with decade buttons, a photos-only toggle and checkbox filters for Type, Platform, Play Style, Connectivity, Players, Origin and Maker, each with live counts. Chips show the chosen filters, with Clear (n) per group and Clear all. Eight games per page, every page number listed, and a clear message with a reset button when nothing matches.
Detail view	A dialog on the same page with the game's photo, all its fields, its description and an RMGP source link. Esc closes it and focus returns to the card.
About	What RMGP is and why preserving early mobile games matters, in my own words.
Collection guide	A chat panel styled like the live site's chat. See the Chatbot section.



The From the Archive filters. Every option and count is built from the games returned by the API.

Every filter option comes from the data itself. A group with no options left disappears, as on rmgd.org, and a ticked option stays listed even when it now matches nothing, so it can be unticked. The layout is mobile-first: at 375px wide it has no sideways scrolling, and the filter groups and cards stack into one column. All controls work with the keyboard.

How it fits together



The page sends its filters to the Lambda as query parameters and draws whatever comes back. During development, Vite forwards /api to the Function URL, so the browser only talks to localhost and the URL stays out of the page code. The same JSON file seeds the table, and the API tests check every stored item against it.

Data

The records come from RMGP's public games API (311 items) and are used for this assignment only.

- **45 games, not 15 to 20.** I took 10 per decade from the 1970s to the 2000s and all 5 from the 2010s, and skipped the one record with no year. I went above the brief on purpose so that every filter has several values to demonstrate.
- **Chosen for coverage.** I applied the same rules rmgd.org uses to build its filters, then picked games that cover as many filter values as possible.
- **Same 22 fields on every record:** id, title, year, yearLabel, decade, device, type, platform, hardware, genre, playStyle, connectivity, players, developer, maker, country, origin, description, image, sourceUrl, featuredRank, sk.
- **Cleaned like the live site.** For example, 'USA' and 'United States of America' both become 'United States'. device is RMGP's Platform label, or its Type when the Platform is empty.
- **Images** are shown from RMGP's own image store rather than downloaded into the repository. 11 records have no photo, which is what the photos-only toggle hides.

DynamoDB design

Setting	Value
Table	rmgp-games
Region	us-east-2 (Ohio)
Capacity	On-demand, personal account, free tier
Partition key	decade (String), for example 1980s
Sort key	sk (String) = device#id, for example Game Boy#bca605de
Items	45, the same records as games.json

Why these keys

- One device and one decade, the access pattern in the brief, is a single Query: decade = 1980s and the sort key begins with 'Game Boy#'.
- The decade partitions are even for this data (10, 10, 10, 10 and 5). With device as the partition key, one partition would hold 15 games and many would hold one.
- The id at the end of the sort key keeps two games on the same device unique.
- A year range runs one Query per decade it touches, at most five, and the Lambda trims to the exact years. A device-only request Scans the table, which is fine at 45 items.

One device and one decade

```
aws dynamodb query --table-name rmgp-games --region us-east-2 \
  --key-condition-expression "decade = :d AND begins_with(sk, :device)" \
  --expression-attribute-values '{":d":{"S":"1980s"},":device":{"S":"Game Boy#"}}'
```

Through the API, the same lookup is GET /?decade=1980s&device=Game%20Boy.

At a larger scale

DynamoDB is built for fast key lookups, not free-text search or many combined filters. With thousands of games I would add secondary indexes for the most common single-field lookups, move keyword and multi-filter search to a search service such as OpenSearch, and choose a partition key with more values than five decades.

API and security

The Lambda reads q, sort, year_min, year_max, decade, photos, type, device, play_style, connectivity, players, origin, maker and page. Values inside one filter are combined with OR, and different filters with AND. The keyword search covers the title, developer, country and description, and ignores case and accents, so 'pokemon' finds 'Pokémon'. Each response returns the page of games, the totals, live counts for every filter, and the year span and decades for the slider.

- No AWS keys anywhere in the repository, the screenshots or the page code.
- The table is read through an inline policy that allows only dynamodb:Query and dynamodb:Scan on this one table.
- The Function URL can only read public RMGP data. It is shared with reviewers only and kept out of the repository, and an AWS budget alert is set.
- The table, the Lambda and its URL will be deleted after the interview.

Chatbot

The collection guide uses **no AI model and no keys**, so it cannot invent a game. It reads each question for decades, years and year ranges, two-player or one-player wording, 'with photos', and any filter value that exists in the collection, plus a few everyday words such as 'phone' or 'japanese'. Any other words become a title search. It then calls the same API as the archive and replies only from what comes back.

Question	Answer from the live data
Which games are for two players?	Lists the 9 matching games, 8 at a time, and points to the filters for the rest
How many games are from Japan?	There are 16 games from Japan.
Who made Snake?	The full record: 1997, by Nokia, from Finland, on Nokia phones, single-player
Is Tetris in the collection?	That isn't in this collection.
lemings	That isn't in this collection. Did you mean: Lemmings

It does not understand follow-up questions, and it is not open conversation.

Testing

- **90 page tests** (Vitest and Testing Library) run against a fake API that follows the Lambda's rules. They cover loading and errors, filter options and counts, every filter, decades and the year slider, search, sort, the photos toggle, chips and clear links, paging, the detail dialog, the hero, the navbar, the About section, the chatbot and the page order.

- **72 API checks** run against the live Lambda and table. They confirm that every stored item matches games.json, the keys follow the design, every filter and combination returns the right games, counts are correct, paging is complete, and bad input gets a clear error.
- The page tests run in a simulated browser, so layout and animation were checked by hand in Chrome.

Choices that differ from the brief

Brief	What I did, and why
15 to 20 games	45 games, so every filter has enough values to demonstrate.
Download the images	Shown from RMGP's image store, credited, rather than copied into the repository.
Images section	Not built in the time available.
Section order	The archive comes straight after the hero, as on the live homepage. The live site's Discover & Explore section is left out.
Plain page	Built with React (Vite), so the filters, dialog and chat are separate, testable components.

What I would do next

- Add the Images section, and download the images into the project as the brief asks.
- Host the built page on S3 and CloudFront, with CORS on the Function URL or API Gateway with a usage limit.
- Give the Lambda its own execution role with only the two read actions, in place of the broader role it uses today.
- Let the chatbot handle follow-up questions, apply its filters to the archive, and open a game's detail view from its answers.
- Add secondary indexes or a search service if the collection grows, and browser end-to-end tests for layout and scrolling.

Credits

- Game data, photos and the hero video: Retro Mobile Gaming Project (rmgd.org), used for this assignment only. The page's layout and colours follow rmgd.org.
- Fonts: Geist and Geist Mono, from Fontsource.
- Built with help from Claude Code, an AI coding assistant, which also helped draft the About text.